

Technology Stack & Application Characteristics

Proposed implementation stack for a lightweight, maintainable incident automation system

DATE	STUDENT NAME	PROJECT
07 October 2026	Gangadevi.R	Incident Life Cycle Automation

Note The stack below is a project-level proposed stack. Replace individual technologies only if your actual implementation differs.

Component	Purpose	Proposed Technology
User Interface	Incident portal, dashboards and role-based screens	HTML, CSS, JavaScript
Application Logic	Incident lifecycle, validation, routing and status transitions	Python + Flask
Automation Engine	SLA timers, scheduled checks and escalation rules	Python scheduler / background worker
Database	Incidents, users, rules, status history and audit records	SQLite for prototype / MySQL for deployment
Notifications	Assignment, status and SLA alerts	SMTP / Email service
API Layer	Internal REST-style endpoints between UI and backend	Flask API routes
File / Evidence Storage	Optional incident attachments and evidence	Local storage or object storage
Infrastructure	Application runtime and deployment	Local server / cloud VM

Application Characteristics

Characteristic	Description	Technology / Approach
Open-source frameworks	Prefer maintainable, widely used components	Python, Flask, JavaScript
Security	Authentication, role checks, input validation, secure sessions, least privilege	Flask security practices / HTTPS in deployment
Scalable architecture	Separate UI, application logic, data and automation responsibilities	Layered web architecture

Characteristic	Description	Technology / Approach
Availability	Usable during normal operation and recoverable after restart.	Service restart + backup
Performance	Fast incident lookup and lightweight automation checks; avoid blocking user requests	Indexed queries + background scheduler

Reference Concepts

- Layered application architecture and clear separation of concerns.
- REST-style service boundaries for maintainability.
- Role-based access and audit logging for operational traceability.